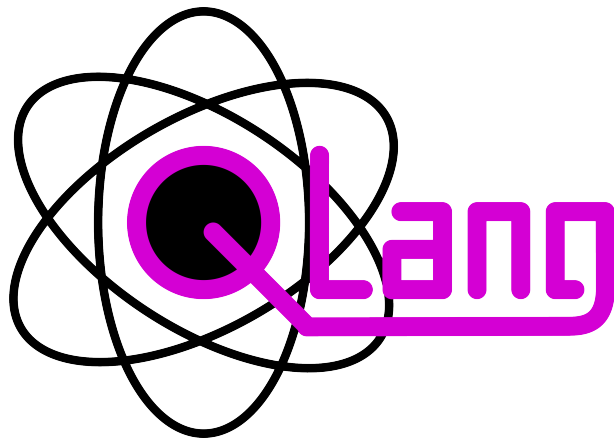




QLang Programming Language

Language Specification & Documentation

Created by:

Kamil Widzyk
Maciej Świątek
Jan Wida

Contents

1	Introduction	3
1.1	Overview	3
1.2	Quick Example	3
1.3	Design Principles	3
1.3.1	Classical and Quantum Code in One Language	3
1.3.2	Structured Error Reporting	3
1.3.3	Modular Structure via Places	4
2	Language Basics	5
2.1	Program Structure	5
2.2	Data Types	5
2.3	Variables	6
2.3.1	Declaration Syntax	6
2.3.2	Arrays	6
2.3.3	Constants	6
2.4	Operators	6
2.4.1	Arithmetic Operators	6
2.4.2	Unary Operators	7
2.4.3	Comparison and Logical Operators	7
2.4.4	Increment and Decrement	7
2.4.5	Compound Assignment Operators	7
2.4.6	Assignment Expression	7
2.4.7	The <code>#</code> Size Getter	7
2.4.8	The <code>reset</code> Operator	8
2.4.9	String Formatting Operator	8
2.4.10	The Parent Operator <code>^</code>	8
2.5	Control Flow	8
2.5.1	Conditional Statements	8
2.5.2	For Loop	9
2.5.3	While Loop	9
2.5.4	Iterate Loop	9
2.5.5	Break and Continue	10
2.6	Functions	10
2.6.1	Named Arguments	10
2.6.2	Variadic Parameters	10
2.7	Input and Output	11
2.7.1	Output	11
2.7.2	Input	11

3	Quantum Computing	12
3.1	Quantum Types: state and obs	12
3.2	Primitive Gates	12
3.3	Measurement	12
3.4	Example: Quantum Teleportation	13
3.5	Example: Superdense Coding	14
4	Advanced Control Flow and Scope Management	15
4.1	The Parent Operator ^ and Scope Hierarchy	15
4.1.1	Scope in QLang	15
4.1.2	How ^ Works	15
4.2	Advanced Array Structures and Indexing	15
4.2.1	Simple Indexing and Negative Indices	16
4.2.2	Range Indexing	16
4.2.3	List Indexing	16
4.3	Scope Lifetime	16
4.4	Scope Interaction with const	16
4.5	Functions and Closures	17
5	Places and Inter-Place Communication	18
5.1	Place Declaration	18
5.1.1	The Global Place	18
5.1.2	Functions Inside Places	18
5.2	Console and I/O per Place	18
5.2.1	show_console(0/1)	18
5.3	Sending Data: send	19
5.4	Receiving Data: receive	20
5.4.1	Blocking Behaviour	20
5.4.2	Array Receives	20
5.5	Checking Availability: available	21
5.6	Packet Matching Rules	21
5.7	Packet Logging: packet_log()	22
5.8	Example: Distributed Prime Checker	23

1 Introduction

QLang is a high-level programming language designed for quantum computing and distributed quantum networking. It provides a simple and intuitive syntax for expressing quantum algorithms, operations, and communication between separate quantum nodes called places, making it accessible to both beginners and experienced programmers in the field of quantum computing.

1.1 Overview

A QLang program is made up of top-level items: **place** declarations, function declarations, import directives, and statements. A place is QLang's primary unit of concurrent execution. Each place has its own scope, defines its own functions, and runs as a separate process. Code written outside any named place belongs to the implicit **global** place, which only executes if it contains at least one statement or function declaration.

The language supports five built-in variable types: **state**, **obs**, **num**, **text**, and **any**. Quantum-specific operations, including the **measure** keyword and all built-in gates, are first-class constructs in QLang, not library calls. The language natively supports quantum gates such as **superpose**, **shift**, **not**, **dual_not**, **phase_not**, **entangle**, **entangle_phase**, and **swap**. QLang is interpreted. Source files with the extension **.ql** are parsed by an ANTLR4-generated parser and executed by the QLang interpreter written in Python.

1.2 Quick Example

The following complete QLang program creates a Bell state (a maximally entangled two-qubit state), measures both qubits, and prints the results:

```
place Alice {
    state a, b;

    superpose a;           // Put qubit 'a' into superposition
    entangle a -> b;       // Entangle 'a' with 'b' to create a Bell state

    obs ma = measure a;
    obs mb = measure b;

    println("Measurement A: %d" % ma);
    println("Measurement B: %d" % mb);
}
```

After running, both measurements will always agree, both **0** or both **1**, which is the defining property of a Bell state.

1.3 Design Principles

1.3.1 Classical and Quantum Code in One Language

QLang does not separate quantum and classical concerns into different APIs or files. A variable of type **obs** and a variable of type **state** (qubit) can appear side-by-side in the same scope. Measurement, when invoked on a **state**, collapses the qubit and returns an **obs** value indicating the outcome: **F** (false) if the qubit collapsed to $|0\rangle$, and **T** (true) if it collapsed to $|1\rangle$.

1.3.2 Structured Error Reporting

When the interpreter encounters an error, whether a syntax mistake, an undefined variable, or an illegal runtime operation, it displays a detailed error box that highlights the exact source location, shows surrounding context lines, and explains the cause.

1.3.3 Modular Structure via Places

QLang organizes code into units called **places**. Each **place** has its own name, its own variables, and runs independently from other places. Places can communicate with each other using the built-in **send** and **receive** instructions, which transfer both classical and quantum data. This makes it natural to write distributed programs, such as quantum communication protocols, directly in QLang.

2 Language Basics

This chapter covers the fundamental building blocks of QLang: how programs are structured, the available data types, variables, operators, and control flow constructs.

2.1 Program Structure

A QLang source file (extension `.ql`) consists of a sequence of top-level items. Each item is one of the following:

- A **place** declaration.
- A function declaration (at global place).
- A statement (at global place).
- An import directive, which supports two styles:
 - `<< "path" >>` – a lexical splicing directive that literally splices the content of the specified file at the current location before parsing.
 - `<< "path" >>` – a runtime function import directive that dynamically loads and registers only the function definitions from the specified file during execution.

All named places start executing simultaneously when the program is launched. The implicit global place is only executed if it contains at least one statement or function declaration.

2.2 Data Types

QLang has five built-in variable types, designed to cover both classical and quantum computation:

Type	Description
state	A quantum state (qubit or qubit register). Can be in superposition and entangled with other qubits. The stored quantum information is only accessible via the measure keyword; states cannot be read, assigned, or printed directly.
obs	A classical observation. A scalar obs stores a boolean value; a sized register obs x[N] stores an N-bit integer and allows access to individual bits by index. Used for branching, control, and classical post-processing after measurement.
num	A universal numeric type. It dynamically handles both integers and floating-point numbers, automatically adapting its internal representation based on the assigned value and performed operations. Used for all classical numerical computations.
text	A string of characters. Used for textual data, print formatting, and string operations. Supports the same indexing modes as arrays.
any	A generic type usable in function parameter declarations and variable declarations. A parameter of type any accepts a value of any type and infers the actual type at the call site.

2.3 Variables

Variables are declared by specifying a type, a name, optional size specifiers and an optional initial value.

2.3.1 Declaration Syntax

```
num pi = 3.14159;           // floating-point number
num count = 10;             // num accepts integer values too
obs result[16];             // 16-bit integer register, default value 0
obs counter[8] = 0;         // 8-bit integer register, initialized to 0
text greeting = "Hello!";   // string
```

2.3.2 Arrays

Any variable type can be declared as an array by appending one or more size specifiers in square brackets. Arrays are zero-indexed. A size can be a fixed integer literal or an expression evaluated at runtime. Using `[?]` as the size declares a dynamic array, whose length adjusts automatically when elements are appended using the `+=` operator:

```
num matrix[4][4];           // two-dimensional 4x4 array

// Size inferred automatically from the initialiser
num a[?] = [1, 2, 3, 4, 5]; // size is 5, inferred from the list
println(#a);                // 5

// Runtime-computed size
num n = 8;
num dynamic[n];              // size determined at runtime
```

2.3.3 Constants

The **const** keyword can be applied to any variable declaration to prevent reassignment after initialisation. A variable can also be declared mutable first and then locked with **const x;**, once frozen, further assignments raise a runtime error. Variable of type **state** cannot be declared as constants or frozen. Attempting to apply **const** to a **state** variable will raise a runtime error.

```
const obs MAX_SIZE[8] = 100; // sized obs constant
const num EPSILON = 0.0001;  // num constant

num x = 10;
x++; // still mutable
const x; // freeze: further assignments raise an error
```

2.4 Operators

2.4.1 Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo
**	Exponentiation

2.4.2 Unary Operators

Operator	Description
+x	Unary plus, returns the value unchanged.
-x	Unary minus, negates the value. If applied to a boolean type, behaves as logical NOT and returns the negated boolean value (T or F).
! x	Logical NOT. If x is a boolean type, returns the negated boolean (T or F); otherwise, returns 1 if x is zero, 0 otherwise.

2.4.3 Comparison and Logical Operators

Operator	Description
==	Equal to
!=	Not equal to
<, >, <=, >=	Relational comparisons
&&	Logical AND
 	Logical OR

2.4.4 Increment and Decrement

```
num i = 0;
i++;    // post-increment
++i;    // pre-increment
i--;    // post-decrement
--i;    // pre-decrement
```

2.4.5 Compound Assignment Operators

In addition to plain assignment (**=**), QLang supports compound operators: **+=**, **-=**, ***=**, **/=**, **%=**, ****=**. The operators **&=** and **|=** are logical compound assignments equivalent to **x = x && y** and **x = x || y** respectively.

All compound assignment operators return the new value of the variable, so they can be used inside expressions.

2.4.6 Assignment Expression

Plain assignment (**=**) is also an expression and returns the assigned value. Assignment must always target a variable; attempting to assign to a non-variable expression (e.g. **5 = x**) raises a runtime error.

2.4.7 The # Size Getter

The **#** prefix operator returns the current length of an array or a **text** string. For dynamic arrays, this reflects the actual number of elements after any appends:

```
num items[?] = [10, 20, 30, 40, 50];
println(#items);    // 5
text message = "hello";
println(#message);  // 5
```


2.4.8 The **reset** Operator

The **reset** operator restores a variable to its initial value as declared, or to the type default if no initial value was given. It works on scalars, strings, and multi-dimensional arrays. It returns the value after reset.

```
// reset scalar
num x = 5;
x = 99;
reset x;
println(x); // 5

// reset text
text s = "hello";
s = "world";
reset s;
println(s); // hello

// reset array
num arr[2] = [1, 2];
arr[0] = 99;
reset arr;
println(arr[0]); // 1
```

2.4.9 String Formatting Operator

The **%** operator, when the left operand is a **text** string, performs printf-style interpolation:

```
num x = 42;
num y = 3.14159;
println("The answer is %d" % x);
println("x = %d, y = %.2f" % [x, y]); // list of values

text fmt = "Hello, %s!";
println(fmt % "world");
```

2.4.10 The Parent Operator **^**

The **^** operator explicitly accesses a variable in an outer scope. Multiple carets chain upward, one level per caret. See Section 4.1 for a full description.

2.5 Control Flow

2.5.1 Conditional Statements

QLang supports standard **if/else if/else** branching. **elif** is a shorthand alias for **else if**. When the branch body is a single statement, curly braces may be omitted. An inline ternary form **cond ? a : b** is also available as an expression:

```
num x = 75;

if (x > 100) {
  println("Large");
} elif (x > 50) {
  println("Medium");
} else {
  println("Small");
}

// if without braces
if (x > 0) println("Positive");

// Ternary operator
text label = x > 50 ? "big" : "small";
println(label);
```

2.5.2 For Loop

The **for** loop iterates over a numeric range with an optional step. The syntax is **for <var> from <start> to <end> step <step>**. The range is exclusive. The loop variable is created automatically and is only accessible inside the loop body. When **step** is omitted, the interpreter infers it automatically: **+1** if **end > start**, **-1** otherwise.

```
// ascending for loop
for i from 1 to 5 {
    println(i); // 1 2 3 4
}

// descending for loop
for i from 5 to 1 {
    println(i); // 5 4 3 2
}

// custom step for loop
for i from 0 to 10 step 2 {
    println(i); // 0 2 4 6 8
}
```

2.5.3 While Loop

The **while** loop repeats its body as long as the condition evaluates to **true** for boolean expressions, or is greater than zero for numeric expressions.

```
// while loop
num x = 0;
while (x < 5) {
    println(x);
    x++;
}
// 0 1 2 3 4
```

2.5.4 Iterate Loop

The **iterate** loop traverses every element of an array. The current element is bound to a named variable. An optional **index** clause provides a second variable that holds the current zero-based index. Both **break** and **continue** work inside **iterate**:

```
// iterate over array
num arr[4] = [10, 20, 30, 40];
iterate arr as val {
    println(val); // 10 20 30 40
}

// iterate with index
iterate arr as val index i {
    println("%d: %d" % [i, val]);
    // 0: 10
    // 1: 20
    // 2: 30
    // 3: 40
}

// break and continue work inside iterate
iterate arr as val {
    if (val == 20) continue;
    if (val == 40) break;
    println(val); // 10 30
}
```

2.5.5 Break and Continue

break exits the nearest enclosing loop immediately. **continue** skips the rest of the current iteration and proceeds to the next one.

2.6 Functions

Functions are declared with the **function** keyword. Parameters are typed, can have default values, and may be arrays. Supported parameter types are **num**, **obs**, **state**, **text**, and **any**. A parameter of type **any** accepts a value of any type and infers the actual type at the call site. A function returns a value with the **return** statement.

```
function add(num a, num b) {
  return a + b;
}

function greet(text name = "World") {
  println("Hello, " + name + "!");
}

function power(num base, num exp) {
  return base ** exp;
}

// any parameter accepts any type
function print_value(any x) {
  println(x);
}

greet(); // Hello, World!
greet("Alice"); // Hello, Alice!
println(add(3, 4)); // 7
println(power(exp = 3, base = 2)); // 8
print_value(42); // 42
print_value("hello"); // hello
```

2.6.1 Named Arguments

Arguments can be passed by name, which allows them to be supplied in any order and makes call sites more self-documenting:

```
function rect(num width, num height = 1) {
  return width * height;
}

// Positional arguments
println(rect(5, 3)); // 15

// Mixed: positional + named
println(rect(5, height = 3)); // 15

// All named, any order
println(rect(height = 4, width = 6)); // 24

// Using default value
println(rect(7)); // 7
```

2.6.2 Variadic Parameters

A function can accept a variable number of arguments using the **...** prefix, for example **function sum(...values)**. Variadic arguments do not have a type written before the name and may mix values of different types. Inside the function they are available as an array:

```

function sum(...values) {
    num s = 0;
    for i from 0 to #values {
        s += values[i];
    }
    return s;
}

println(sum(1, 2, 3));           // 6
println(sum(10, 20, 30, 40));   // 100

// Regular parameters can precede the variadic one
function prefixed(num x, ...rest) {
    println("x=" + x + " count=" + #rest);
}

prefixed(5, 10, 15);           // x=5 count=2

```

2.7 Input and Output

2.7.1 Output

QLang provides three output statements:

- **print(...)** — prints values without a trailing newline.
- **println(...)** — prints values followed by a newline.
- **debug(x)** — prints detailed diagnostic information, including the internal type, shape, and for quantum states the gate history and stabilizer representation. To inspect the internal representation of a variable rather than its evaluated value, you can pass it by reference using the **@** prefix operator (e.g., **debug(@x)**).

```

// Basic output
print("no newline");
println(" with newline");
println();           // blank line

// Multiple arguments
println(1, 2, 3);    // 1 2 3

// Printf-style formatting
num val = 42;
println("The answer is %d" % val);
println("x = %d, y = %.2f" % [val, 3.14]);

```

2.7.2 Input

The **input** statement reads a value from the user into a variable. An optional constraint limits the accepted input. For numeric variables the constraint specifies a value range using the notation **min..max**. For **text** variables the constraint specifies the allowed length in characters. The interpreter keeps prompting until a valid value is entered.

```

obs age[8];
input(age);           // read any value

obs score[8];
input(score, 0..100); // accept only values in range [0, 100]

```

3 Quantum Computing

This chapter introduces the quantum-specific features of QLang: the built-in quantum types, the primitive quantum gates available in the language, how measurement works, and two canonical example protocols: quantum teleportation and superdense coding.

3.1 Quantum Types: **state** and **obs**

QLang exposes two first-class types for quantum programming:

- **state**: a quantum register (one or more qubits). A **state** can be put into superposition and entangled with other **states** using the built-in gate primitives. The stored quantum information is only accessible via the **measure** keyword; states cannot be read, assigned, or printed directly.
- **obs**: a classical observation. A scalar **obs** stores a boolean value; a sized register **obs x[N]** stores an N-bit integer and supports index access to individual bits. Used for branching, control, and classical post-processing after measurement.

A variable declaration can specify arrays and bit-widths. **state q[2]** declares a two-qubit register; **obs result[8]** declares an 8-bit classical register.

3.2 Primitive Gates

QLang provides primitive quantum gates as first-class language constructs. Each gate, excluding swap, has a short uppercase name and a lowercase alias. The supported gates are listed below.

Gate	Alias	Description
H	superpose	Hadamard gate, creates superposition on a single qubit.
S	shift	Phase gate, introduces a phase shift on a single qubit.
X	not	Pauli-X, flips the computational basis state.
Y	dual_not	Pauli-Y, combines a bit flip and a phase flip.
Z	phase_not	Pauli-Z, applies a relative phase flip between $ 0\rangle$ and $ 1\rangle$.
CNOT	entangle	Controlled-NOT, entangles two qubits.
CZ	entangle_phase	Controlled-Z, applies a phase flip conditioned on the control qubit.
–	swap	Swap gate, swaps the states of two qubits.

Single-qubit gates act on one **state** variable in-place. Most multi-qubit gates (like **CNOT** or **CZ**) use the arrow syntax ($\rightarrow$) to indicate the control-to-target relationship. However, the **swap** gate has its own syntax and simply takes two space-separated qubits: **swap q1 q2**;

3.3 Measurement

Measurement forces a **state** to collapse into a classical outcome. The **measure** keyword returns an **obs** value indicating the outcome: **F** (false) for $|0\rangle$, and **T** (true) for $|1\rangle$. Measuring an array of qubits at once produces a list of **obs** values, which can be assigned directly to an **obs** register to form a classical number.

QLang also provides **measureX**, which applies a Hadamard gate before measuring, effectively measuring in the X basis rather than the Z basis.

The following example puts a qubit into superposition, measures it, and prints the result:

```

state q;
H q;
obs result = measure q;
println(result);

```

Measurement is nondeterministic when applied to a superposed or entangled state.

3.4 Example: Quantum Teleportation

Quantum teleportation transfers the state of a qubit from one place to another using a shared entangled pair and two classical bits of communication. The sender measures their qubits and sends the classical results; the receiver applies correction gates based on those results to recover the original state.

```

place Alice {
    println("Alice is preparing to teleport a qubit to Bob...");
    // Prepare Bell pair
    state q1, q2;
    H q1;
    CNOT q1 -> q2;
    // Send one qubit to Bob
    send q2 to Bob as "qubit";
    println("Bell pair prepared. One qubit sent to Bob.");

    // Get one bit of input from user
    state to_send;
    num flip;
    print("Enter a bit to teleport (0 or 1): ");
    input(flip, 0..1);
    println("Input '%d' received. Preparing qubit to teleport..." % flip);

    // 0 = no change, 1 = apply X gate
    if (flip == 1) X to_send;

    // Prepare qubit to teleport
    CNOT to_send -> q1;
    H to_send;

    // Measure both
    obs measurements[2] = measure [to_send, q1];

    // Send measurement results to Bob
    println("Measurement complete: m0=%d m1=%d" % [measurements[0], measurements[1]]);
    send measurements to Bob as "measurements";
    println("Measurement results sent to Bob.");
}

place Bob {
    // Receive qubit from Alice (half of Bell pair)
    println("Waiting for qubit from Alice...");
    state q2 = receive state from Alice named "qubit";
    println("Received qubit from Alice.");

    println("Waiting for measurement results from Alice...");
    obs measurements[2] = receive obs from Alice named "measurements";
    println("Received measurement results from Alice.");

    // Apply corrections based on Alice's measurements
    if(measurements[0] == 1) Z q2;
    if(measurements[1] == 1) X q2;

    // q2 should now be in the same state as Alice's original to_send qubit
    println("Teleportation complete.");
    obs orig_state = measure q2;

    println("Original state (0 or 1): %d" % orig_state);
}

```

3.5 Example: Superdense Coding

Superdense coding allows two classical bits of information to be transmitted using only one qubit, provided the sender and receiver share an entangled pair beforehand. The sender encodes the two bits by applying gates to their half of the pair and sends the qubit; the receiver decodes both bits by measuring the pair jointly.

```
place Alice{
    // Prepare entangled pair and send one qubit to Bob
    state qA, qB;
    H qA;
    CNOT qA -> qB;
    send qB to Bob as "qubitB";

    // Get two bits of input from user
    obs bits[2];
    print("Enter number 0-3 (two bits): ");
    input(bits, 0..3);
    println("Input '%d' received. Preparing qubit to send..." % bits);
    println("Bits to send: %d %d" % [bits[0], bits[1]]);

    // Encode the two bits into the qubit
    if (bits[0]) Z qA; // Flip phase for bit 0
    if (bits[1]) X qA; // Flip bit for bit 1

    // Send the encoded qubit to Bob
    send qA to Bob as "qubitA";
    println("Encoded qubit sent to Bob.");
}

place Bob {
    println("Bob is waiting for qubits from Alice...");
    state qB = receive state from Alice named "qubitB";
    println("Received first qubit from Alice.");
    state qA = receive state from Alice named "qubitA";
    println("Received second qubit from Alice.");

    // Decode the received qubits
    CNOT qA -> qB;
    H qA;
    obs result[?] = measure [qA, qB];
    println("Measurement complete. Decoded bits: %d %d" % [result[0], result[1]]);
    println("Decoded number: %d" % result);
}
```

4 Advanced Control Flow and Scope Management

4.1 The Parent Operator ^ and Scope Hierarchy

4.1.1 Scope in QLang

In QLang, every block of code creates a new scope level, forming a linked chain of scopes. Both reading and writing a variable automatically walk this chain upward until the variable is found. The parent operator ^ is needed only when you want to explicitly skip the current scope and target a specific outer level - for example, when a local variable shadows an outer one.

4.1.2 How ^ Works

The parent operator ^ explicitly selects a scope one level above the current one. Multiple carets chain upward, one level per caret:

- ^X accesses X in the parent scope
- ^^X accesses X in the grandparent scope
- ^^^X accesses X three levels up

Once the target scope is reached, the variable is looked up only in that specific scope; the automatic upward walk does not apply at that point. The operator supports all standard operations: assignment, compound assignment, increment and decrement, as well as indexed access on arrays.

The following example demonstrates reading and writing through the scope hierarchy:

```
num x = 1;
if (1) {
    num x = 2;
    if (1) {
        println("%d" % ^x);    // 2 - reads parent x
        ^x = 7;
        println("%d" % ^x);    // 7 - assigns to parent x
        (^x)++;
        println("%d" % ^x);    // 8
        ^x += 3;
        println("%d" % ^x);    // 11
        println("%d" % ^^x);   // 1 - reads grandparent x
    }
}
```

Array indexing works the same way. The parent operator applies to the variable, and the index is evaluated afterward:

```
num arr[2] = [10, 20];
if (1) {
    if (1) {
        println("%d" % ^^arr[0]); // 10
        println("%d" % ^^arr[1]); // 20
    }
}
```

4.2 Advanced Array Structures and Indexing

QLang supports three indexing modes for arrays: simple indexing, range indexing, and list indexing. All three resolve negative indices automatically against the container length.

4.2.1 Simple Indexing and Negative Indices

A single integer index selects one element. Negative values count from the end of the array: an index **i** where **i < 0** is resolved as **i + length** before access.

```
num arr[5] = [10, 20, 30, 40, 50];
println(arr[0]);    // 10
println(arr[-1]);   // 50
println(arr[-2]);   // 40
```

4.2.2 Range Indexing

A range index **start..end** selects a contiguous subarray. The range is left-inclusive and right-exclusive: element at **start** is included, element at **end** is not. Negative values are resolved the same way as in simple indexing.

```
num arr[5] = [10, 20, 30, 40, 50];
println(arr[1..3]); // [20, 30]
println(arr[0..-1]); // [10, 20, 30, 40]
```

4.2.3 List Indexing

A list index **[a, b, c]** selects elements at the given positions in arbitrary order. The double-bracket syntax is used for a literal list of indices. To pass an existing array variable as the index list, use a single bracket. The interpreter detects that the expression is an array and treats it as a list of indices automatically. All three indexing modes also work on **text** strings.

```
num arr[5] = [10, 20, 30, 40, 50];
println(arr[[0, 2, 4]]); // [10, 30, 50]
println(arr[[4, 0, 2]]); // [50, 10, 30]
num idx[3] = [0, 2, 4];
println(arr[idx]); // [10, 30, 50]
```

4.3 Scope Lifetime

In QLang, variables are bound to the specific scope they are declared in and are destroyed as soon as that scope ends. This is particularly important when working with loops.

```
num sum = 0;
for i from 1 to 3 {
  // This variable is created fresh in every iteration
  num temp = i * 10;
  sum += temp;
}
// Trying to access 'temp' or 'i' here would result in an error
```

4.4 Scope Interaction with **const**

The **const** keyword interacts with the scope hierarchy in two distinct ways, depending on whether you are declaring a new variable or modifying an existing one.

If you declare a new constant, it creates a fresh variable in the current scope, potentially shadowing outer variables. However, if you apply **const** to an already existing variable, the interpreter searches up the scope tree to find the nearest **x** and marks that specific instance as constant.

```
num global_val = 10;
num local_val = 20;
```

```
if (T) {  
  // This shadows the outer 'local_val'.  
  const num local_val = 50;  
  
  // Making an existing outer variable constant from within a local scope.  
  const global_val;  
}  
// The outer 'local_val' remains 20 and is NOT constant.  
local_val = 21; // Valid  
  
// global_val = 11; // Not valid, cannot reassign const variable
```

4.5 Functions and Closures

Functions in QLang act as closures. When a function is declared, it captures the exact scope hierarchy present at the moment of its creation (its **closure_scope**).

When the function is later called, the interpreter creates a new execution scope for the function's body. The parent of this new scope is set to the captured closure scope, not the scope from which the function was called.

5 Places and Inter-Place Communication

This chapter describes QLang’s concurrency model: how a program is divided into independently executing **places**, how each place gets its own console, and how places exchange both classical and quantum data using the built-in **send**, **receive**, and **available** constructs.

5.1 Place Declaration

A **place** is QLang’s primary unit of concurrent execution. Each **place** declaration creates an isolated context that runs in its own operating-system process. Places start executing simultaneously when the program is launched.

The syntax is:

```
place Name {  
    // statements and function declarations  
}
```

Name must consist of letters, digits, and underscores. The name **global** is reserved and cannot be used. Place names must be unique within a program. Each place has its own scope, its own variables, and its own set of functions.

The following example declares two places that run in parallel:

```
place Alice {  
    println("Hello from Alice!");  
}  
  
place Bob {  
    println("Hello from Bob!");  
}
```

Both places start at the same time. Their output may interleave in any order.

5.1.1 The Global Place

Code written outside any **place** declaration belongs to the implicit **global** place. The global place behaves like any other place: it runs as a separate process and gets its own console window. If the source file contains no **place** declarations at all, the entire program executes inside the global place.

5.1.2 Functions Inside Places

Functions declared inside a place are local to that place. They cannot be called from another place. Each place can also declare its own variables, constants, and loops without interfering with other places.

5.2 Console and I/O per Place

Every place opens a separate console window when the program starts. The window title includes the place’s name (e.g., **Place: Alice**). All **print**, **println**, and **input** calls from a place are routed to that place’s console window.

5.2.1 **show_console(0/1)**

The **show_console** built-in system function controls the visibility of a place’s console window at run-time. Evaluated as a standard function call rather than a dedicated language keyword, calling **show_con-**

sole(0) hides the window; calling **show_console(1)** makes it visible again. Output produced while the window is hidden is still logged and will be replayed when the window reappears.

This is particularly useful for worker places that perform background computations and do not need a visible console:

```
place Worker {
  show_console(0);    // hide console for this place
  num result = 2 ** 10;
  send result to Main as "result";
  show_console(1);    // show console again before exiting
}

place Main {
  num r = receive num from Worker named "result";
  println("Result: %d" % r);
}
```

5.3 Sending Data: send

The **send** statement transmits a variable from one place to another through the shared packet network. All four QLang variable types can be sent. The general syntax is:

```
send <variable> to <place> as "<name>";
```

Both the **to** and **as** clauses are optional:

Form	Description
<code>send var to Bob as "msg";</code>	Sends the var to Bob with the packet name "msg" . Only Bob can receive it, and the receiver can filter by the name.
<code>send var to Bob;</code>	Sends the var to Bob without a packet name. Only Bob can receive it.
<code>send var as "msg";</code>	Sends the var without a specific target. Any place can receive it, and the receiver can filter by the name "msg" .
<code>send var;</code>	Sends the var without a target or name. Any place can pick it up.

The target place can be specified either as a bare identifier or as a string literal. The following two lines are equivalent:

```
send x to Bob as "data";
send x to "Bob" as "data";
```

The following example shows the different send forms:

```
place Alice {
  num x = 10;
  text msg = "hello";

  // Full form: target and name
  send x to Bob as "number";

  // Target only, no name
  send msg to Bob;

  // Name only, no target, any place can receive
  send x as "data";
}
```

5.4 Receiving Data: **receive**

The **receive** construct declares a new variable and blocks execution until a matching packet arrives from the network. The general syntax is:

```
<type> <name>[size] = receive [<type>] [from <place>] [named "<name>"];
```

The **receive** keyword accepts zero or more optional filters:

Filter	Description
type	An optional type decoration. Note that this filter is syntactically ignored by the interpreter at runtime . Packets are instead filtered based strictly on the declared type of the target variable on the left-hand side of the declaration (which determines the quantum vs. classical packet flag).
from place	Accept only packets sent from the specified place. The place can be given as a bare identifier or a string literal.
named "name"	Accept only packets that were sent with the matching as name.

Filters can be combined in any order. If no filters are specified, the receive accepts the first packet whose type and size match the declared variable:

```
place Alice {  
    num x = 99;  
    send x to Bob;  
}  
  
place Bob {  
    num x = receive;  
    println("Got: %d" % x);  
}
```

A more specific receive using all filters:

```
place Sensor {  
    num reading = 37;  
    send reading to Hub as "temperature";  
}  
  
place Hub {  
    // Filter by type, source, and name  
    num temp = receive num from Sensor named "temperature";  
    println("Temperature: %d" % temp);  
}
```

5.4.1 Blocking Behaviour

The **receive** call is blocking: the place suspends execution until a matching packet is available. If no matching packet ever arrives, the place will wait indefinitely. This is the intended behavior for synchronization between places.

5.4.2 Array Receives

When receiving an array, the size must be declared with the variable. For example, **num data[3] = receive num from Alice named "data";** expects a packet containing exactly three numeric values.

5.5 Checking Availability: **available**

The **available** expression checks whether a matching packet exists in the network without consuming it or blocking. It returns a boolean value that can be used in conditions. The syntax is:

```
available [<type>] [from <place>] [named "<name>"]
```

All filter arguments are optional. Possible filters are:

Filter	Description
type	Check only for packets of the given type (state , obs , num , or text).
from place	Check only for packets sent from the specified place.
named "name"	Check only for packets with the given name.

Unlike **receive**, **available** is non-blocking: it returns immediately with **T** (true) or **F** (false). The packet remains in the network and still needs to be consumed by a **receive** call. A common pattern is to combine **available** with a polling loop:

```
place Alice {
  for i from 0 to 5 {
    num x = (i * 19) % 13;
    send x to Bob as "number";
  }
  text status = "done";
  send status to Bob as "status";
}

place Bob {
  function receive_numbers() {
    num numbers[5];
    for i from 0 to 5 {
      num x = receive num from Alice;
      numbers[i] = x;
    }
    return numbers;
  }

  obs running = T;

  while(running){
    if (available text named "status"){
      text status = receive text named "status";
      println(receive_numbers());
      running = F;
    }
  }
}
```

In this example, Bob continuously checks whether a "**status**" packet has arrived. Once it has, Bob knows Alice has finished sending numbers and can safely receive them all.

5.6 Packet Matching Rules

The QLang packet network uses a set of matching rules to decide which receiver gets which packet. Every packet carries the following metadata:

Field	Description
src_id	Name of the place that sent the packet. Always set automatically.
target_id	Name of the intended recipient place. Set only if the sender used to .
msg_id	Packet name. Set only if the sender used as .
quantum	Whether the packet carries quantum data (state) or classical data.
size	Number of elements in the data (1 for scalar values, array length for arrays).

A packet matches a **receive** request when all of the following conditions are satisfied:

1. **Type and size.** In the network layer, QLang only enforces a binary distinction between classical data (**num**, **text**, **obs**) and quantum data (**state**). It does **not** validate classical subtypes. If a place sends a **text** value and another place attempts to **receive** it as a **num** of the same size, the packet is successfully delivered and the raw text value is assigned to the numeric variable. This bypasses classical type-checking and can lead to type pollution in the receiving scope.
The **size** (number of elements) must match the declared size of the receiving variable exactly.
2. **Target.** If the sender specified a **target_id** (using **to**), then the receiver's place name must equal that **target_id**. If the sender did not specify a target, any place can receive the packet.
3. **Source.** If the receiver specified a **from** filter, then the packet's **src_id** must match. If the receiver did not specify **from**, packets from any source are accepted.
4. **Name.** The **msg_id** matching rule has four cases:
 - Both sender and receiver specified a name: the names must be equal.
 - Neither sender nor receiver specified a name: condition is satisfied.
 - Sender specified a name, receiver did not: condition is satisfied (the receiver accepts any name, including named packets).
 - Sender did not specify a name, receiver did: condition fails (the receiver is looking for a specific name that the packet does not have).

When multiple packets match, the oldest one (the first to arrive) is consumed.

5.7 Packet Logging: **packet_log()**

The **packet_log** built-in system function enables or disables diagnostic logging for network activity in the current place. It is evaluated as a standard function call at runtime rather than a dedicated keyword. Calling **packet_log(1)** turns logging on; **packet_log(0)** turns it off. When enabled, every **send** and **receive** operation in that place prints detailed information to the internal log, including the packet name, source, target, data size, and sequential packet identifier. This is useful for debugging communication patterns between places.

```
place Alice {
  packet_log(1); // enable packet logging
  num x = 42;
  send x to Bob as "value";
  packet_log(0); // disable packet logging
}

place Bob {
  packet_log(1);
  num x = receive num from Alice named "value";
  println("Got: %d" % x);
}
```

5.8 Example: Distributed Prime Checker

The following example demonstrates a manager-worker pattern where a **Manager** place distributes tasks to four **Worker** places and collects results. Each worker checks whether a number is prime and sends back the result. To avoid duplicating code, the actual worker logic is written in a separate file (`worker_code.ql`) and included using the `<< "...">>` operator.

The main program (`worker_prime.ql`):

```
place Manager{
    // Manager: dispatch numbers to workers and collect prime results.
    num max_number = 1000; // change as needed
    num primes[?]; // list for collecting primes
    num current_number = 2; // start checking from 2
    // places that will be run as workers
    text workers[?] = ["Worker1", "Worker2", "Worker3", "Worker4"];

    // initial dispatch: send one task to each worker
    iterate workers as worker{
        send current_number to worker as "task";
        current_number++;
    }

    num runningTasks = #workers; // number of tasks currently running
    num next_worker = 0; // this ensures tasks are split evenly

    while (runningTasks > 0) {
        // collect the result
        num result[2] = receive num named "result"; // [number, isPrime]
        println("Received: %d -> %d" % result);
        if (result[1] == 1) primes += result[0];

        // start new task
        if (current_number <= max_number) {
            text target = workers[next_worker];
            send current_number to target as "task";
            current_number++;
        } else {
            runningTasks--;
        }

        next_worker = (next_worker + 1) % #workers;
    }

    println("Primes found:");
    iterate primes as prime{
        print("%d " % prime);
    }
    println("\nTotal: %d primes." % #primes);

    // Send shutdown signal to workers
    num shutdown_signal = -1;
    iterate workers as worker{
        send shutdown_signal to worker as "task";
    }

    println("Shutdown signals sent to workers. Manager exiting.");
}

// 4 workers, each gets the same code to run
place Worker1{
    <<"worker_code.ql">>;
}

place Worker2{
    <<"worker_code.ql">>;
}

place Worker3{
    <<"worker_code.ql">>;
}

place Worker4{
```



```
<<"worker_code.q1">>;  
}
```

And the corresponding `worker_code.q1` file that is included by each worker:

```
// Receives a number to check from the Manager and sends back [number, isPrime]  
show_console(0); // hide console for workers  
  
// return 1 if n is prime, else 0  
function is_prime(num n) {  
    num i = 2;  
    while (i**2 <= n) {  
        if (n % i++ == 0) return 0;  
    }  
    return 1;  
}  
  
num processed;  
  
// worker loop that runs until a shutdown signal is received  
while (T) {  
    num task = receive num from Manager named "task";  
    if(task == -1) break; // stop loop at shutdown signal  
  
    println("Worker received task: %d" % task);  
    num prime = is_prime(task);  
    num result[2] = [task, prime];  
    println("Worker sending result: %d -> %d" % [task, prime]);  
    send result to Manager as "result";  
    processed++;  
}  
println("Worker processed %d tasks." % processed);  
println("Worker received shutdown signal. Exiting.");  
show_console(1); // show the console again before exiting
```

The Manager dispatches numbers to workers in round-robin fashion. Each worker runs in a loop, receiving tasks, computing results, and sending them back. When the maximum number is reached, the Manager sends a shutdown signal (`-1`) to each worker. The **send/receive** pair with **named "task"** and **named "result"** ensures that task packets and result packets are never confused.